

Unit – 4 Database Cursors [5 Hrs]

PL/SQL Cursor

When an SQL statement is processed, Oracle creates a memory area known as context area. A cursor is a pointer to this context area. It contains all information needed for processing the statement. In PL/SQL, the context area is controlled by Cursor.

A cursor contains information on a select statement and the rows of data accessed by it.

A cursor is used to referred to a program to fetch and process the rows returned by the SQL statement, one at a time. There are two types of cursors:

- Implicit Cursors
- Explicit Cursors

PL/SQL Implicit Cursors

The implicit cursors are automatically generated by Oracle while an SQL statement is executed, if you don't use an explicit cursor for the statement.

These are created by default to process the statements when **DML statements like INSERT, UPDATE, DELETE, SELECT etc. are executed.**

Oracle provides some attributes known as Implicit cursor's attributes to check the status of DML operations. Some of them are: %FOUND, %NOTFOUND, %ROWCOUNT and %ISOPEN.

For example: When you execute the SQL statements like INSERT, UPDATE, DELETE then the cursor attributes tell whether any rows are affected and how many have been affected. If you run a SELECT INTO statement in PL/SQL block, the implicit cursor attribute can be used to find out whether any row has been returned by the SELECT statement. It will return an error if there no data is selected.

The following table specifies the status of the cursor with each of its attribute.

Attribute	Description
%FOUND	Its return value is TRUE if DML statements like INSERT, DELETE and UPDATE affect at least one row or more rows or a SELECT INTO statement returned one or more rows. Otherwise it returns FALSE.
%NOTFOUND	Its return value is TRUE if DML statements like INSERT, DELETE and UPDATE affect no row, or a SELECT INTO statement return no rows. Otherwise it returns FALSE. It is a just opposite of %FOUND.
%ISOPEN	It always returns FALSE for implicit cursors, because the SQL cursor is automatically closed after executing its associated SQL statements.
%ROWCOUNT	It returns the number of rows affected by DML statements like INSERT, DELETE, and UPDATE or returned by a SELECT INTO statement.

Consider the following table:

ID	NAME	SALARY
1	Ram	20000
2	Shyam	22000
3	Hari	24000
4	Sita	26000
5	Gita	28000

Let's execute the following program to update the table and increase salary of each customer by 5000. Here, SQL%ROWCOUNT attribute is used to determine the number of rows affected:

```
DECLARE
    total_rows number(2);
BEGIN
    UPDATE customers
    SET salary = salary + 5000;
    IF sql%notfound THEN
        dbms_output.put_line('no customers updated');
    ELSIF sql%found THEN
        total_rows := sql%rowcount;
        dbms_output.put_line( total_rows || ' customers updated ');
    END IF;
END;
/
```

Output:

```
5 customers updated
PL/SQL procedure successfully completed.
```

PL/SQL Explicit Cursors

The explicit cursors are defined by the programmers to gain more control over the context area. These cursors should be defined in the declaration section of the PL/SQL block. It is created on a SELECT statement which returns more than one row.

Following is the syntax to create an explicit cursor:

```
CURSOR cursor_name IS select_statement;
```

Steps for using Explicit CURSOR in PL/SQL:

1. Declare the cursor to initialize in the memory.
2. Open the cursor to allocate memory.
3. Fetch the cursor to retrieve data.
4. Close the cursor to release allocated memory.

1) Declare the cursor:

It defines the cursor with a name and the associated SELECT statement.

```
CURSOR name IS  
SELECT statement;
```

2) Open the cursor:

It is used to allocate memory for the cursor and make it easy to fetch the rows returned by the SQL statements into it.

```
OPEN cursor_name;
```

3) Fetch the cursor:

It is used to access one row at a time. You can fetch rows from the above-opened cursor as follows:

```
FETCH cursor_name INTO variable_list;
```

4) Close the cursor:

It is used to release the allocated memory. The following syntax is used to close the above-opened cursors.

```
Close cursor_name;
```

PL/SQL Explicit Cursor Example

Consider the following employee table with data:

EID	NAME	SALARY
101	Ram	25000
102	Shyam	55000
103	Gita	34000

```

DECLARE
    --declaring variables
    eid employee.eid%TYPE;
    ename employee.name%TYPE;
    esalary employee.salary%TYPE;
    --declaring cursor
    CURSOR mycursor IS
        SELECT * FROM employee;
BEGIN
    --opening cursor
    OPEN mycursor;
    --using loop to access data
    LOOP
        --fetching data from cursor
        FETCH mycursor INTO eid,ename,esalary;
        --displaying data
        DBMS_OUTPUT.PUT_LINE('Eid: ' || eid || ' Name: ' ||
                               ename || ' Salary: ' || esalary);
        --exit condition
        EXIT WHEN mycursor%NOTFOUND;
    END LOOP;
END;
/

```

Output:

Statement processed.
 Eid: 101 Name: Ram Salary: 25000
 Eid: 102 Name: Shyam Salary: 55000
 Eid: 103 Name: Gita Salary: 34000
 Eid: 103 Name: Gita Salary: 34000

Fetching Cursor Into Row

```

DECLARE
    myrow employee%ROWTYPE;
    CURSOR cursor1 IS
        SELECT * FROM employee;
BEGIN
    OPEN cursor1;
    LOOP
        FETCH cursor1 INTO myrow;
        EXIT WHEN cursor1%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(myrow.eid || ' ' || myrow.name
                               || ' ' || myrow.salary);
    END LOOP;
    CLOSE cursor1;
END;
/

```

Output:

Statement processed.
 101 Ram 25000
 102 Shyam 55000
 103 Gita 34000

PL/SQL Cursor FOR LOOP

The cursor FOR LOOP statement is an elegant extension of the numeric FOR LOOP statement.

The numeric FOR LOOP executes the body of a loop once for every integer value in a specified range. **Similarly, the cursor FOR LOOP executes the body of the loop once for each row returned by the query associated with the cursor.**

A nice feature of the cursor FOR LOOP statement is that it allows you to **fetch every row from a cursor without manually managing the execution cycle i.e., OPEN, FETCH, and CLOSE.**

The cursor FOR LOOP implicitly creates its loop index as a record variable with the row type in which the cursor returns and then opens the cursor.

In each loop iteration, the cursor FOR LOOP statement fetches a row from the result set into its loop index. If there is no row to fetch, the cursor FOR LOOP closes the cursor.

Syntax of the cursor FOR LOOP statement:

```
FOR record IN cursor_name
LOOP
    process_record_statements;
END LOOP;
```

Where, record defines an individual row from cursor.

The following example declares an explicit cursor and uses it in the cursor FOR LOOP statement.

```
DECLARE
    --declaring cursor
    CURSOR mycursor IS
        SELECT eid,name FROM employee;
BEGIN
    --using for loop to access cursor data
    FOR myrecord IN mycursor
    LOOP
        DBMS_OUTPUT.PUT_LINE('Eid: '|| myrecord.eid ||
                               ' Name: '|| myrecord.name);
    END LOOP;
END;
/
```

Output:

```
Statement processed.
Eid: 101 Name: Ram
Eid: 102 Name: Shyam
Eid: 103 Name: Gita
```

Cursor FOR LOOP with a SELECT statement

The following example is equivalent to the example above but uses a query in a cursor FOR LOOP statement.

```
BEGIN
  --using for loop to access cursor data
  FOR myrecord IN (SELECT * FROM employee
                   ORDER BY eid DESC)
  LOOP
    DBMS_OUTPUT.PUT_LINE('Eid: '|| myrecord.eid ||
                        ' Name: '|| myrecord.name);
  END LOOP;
END;
/
```

Output:

```
Statement processed.
Eid: 103 Name: Gita
Eid: 102 Name: Shyam
Eid: 101 Name: Ram
```

PL/SQL Cursor with Parameters

An explicit cursor may accept a list of parameters. Each time you open the cursor, you can pass different arguments to the cursor, which results in different result sets.

Cursors with parameters are also known as parameterized cursors.

The following shows the syntax of a declaring a cursor with parameters:

```
CURSOR cursor_name (parameter_list)
IS
cursor_query;
```

To open a cursor with parameters, you use the following syntax:

```
OPEN cursor_name (value_list);
```

Example of parameterized cursor:

```
DECLARE
eid employee.eid%TYPE;
ename employee.name%TYPE;
--declaring parameterized cursor
CURSOR mycursor(empid INT)
IS
SELECT eid,name FROM employee
WHERE eid=empid;
BEGIN
--passing value
OPEN mycursor(102);
LOOP
FETCH mycursor INTO eid,ename;
EXIT WHEN mycursor%NOTFOUND;
DBMS_OUTPUT.PUT_LINE('Eid: ' || eid ||
' Name: ' || ename);
END LOOP;
END;
/
```

Output:

Statement processed.
Eid: 102 Name: Shyam

Example of parameterized FOR LOOP Cursor:

```
DECLARE
CURSOR mycursor(minval NUMBER, maxval NUMBER)
IS
SELECT * FROM employee
WHERE salary BETWEEN minval AND maxval;
BEGIN
--passing value in for loop
FOR myrecord IN mycursor(20000,40000)
LOOP
DBMS_OUTPUT.PUT_LINE('Name: ' || myrecord.name ||
' Salary: ' || myrecord.salary);
END LOOP;
END;
/
```

Output:

Statement processed.
Name: Ram Salary: 25000
Name: Gita Salary: 34000

PL/SQL parameterized cursor with default values

```
CURSOR cursor_name (  
    parameter_name datatype := default_value,  
    parameter_name datatype := default_value,  
    ...  
) IS  
    cursor_query;
```

Example is shown below:

```
DECLARE  
--parameterized cursor with default value  
CURSOR mycursor(empid INT:= 102)  
IS  
    SELECT * FROM employee  
    WHERE eid=empid;  
BEGIN  
    --not passing any parameter value  
    FOR myrecord IN mycursor  
    LOOP  
        DBMS_OUTPUT.PUT_LINE('Eid: ' || myrecord.eid ||  
                               ' Name: ' || myrecord.name);  
    END LOOP;  
END;  
/
```

Output:

```
Statement processed.  
Eid: 102 Name: Shyam
```


Nested Cursors

In PL/SQL we can create multiple cursors. We can open a cursor inside another cursor. So, we can define nested cursor as a cursor that is opened and operated inside another cursor.

Nested cursor can be used to select data from multiple tables that are in a relationship with each other. We can also use value accessed by outer cursor in inner cursor.

Syntax of Nested Cursor,

```
DECLARE
    CURSOR cursor1 IS (SELECT statement);
    CURSOR cursor2 IS (SELECT statement);
BEGIN
    OPEN cursor1;
    LOOP
        FETCH cursor1 INTO variable_list;

        OPEN cursor2;
        LOOP
            FETCH cursor2 INTO variable_list;
            EXIT condition of cursor2;
        END LOOP;

        EXIT condition of cursor1;
    END LOOP;
END;
```

Example of Nested Cursor:

Consider following tables:

Employee Table

EID	NAME	SALARY	DEPT_ID
101	Ram	25000	1
102	Shyam	55000	2
103	Gita	34000	1

Department Table

DEPT_ID	DEPT_NAME
1	Finance
2	IT

```

DECLARE
    did department.dept_id%TYPE;
    dname department.dept_name%TYPE;
    CURSOR cursor1 IS
        SELECT dept_id,dept_name FROM department;
    CURSOR cursor2 IS
        SELECT name,salary FROM employee WHERE dept_id=did;
    ename employee.name%TYPE;
    esalary employee.salary%TYPE;
BEGIN
    OPEN cursor1;
    LOOP
        FETCH cursor1 INTO did,dname;
        EXIT WHEN cursor1%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE('Department: ' || dname);

        OPEN cursor2;
        LOOP
            FETCH cursor2 INTO ename,esalary;
            EXIT WHEN cursor2%NOTFOUND;
            DBMS_OUTPUT.PUT_LINE(ename || ' ' || esalary);
        END LOOP;
        CLOSE cursor2;
    END LOOP;
    CLOSE cursor1;
END;
/

```

Output:

```

Statement processed.
Department: IT
Shyam 55000
Department: Finance
Ram 25000
Gita 34000

```

Example of Nested Cursor inside FOR Loop

```
DECLARE
  did department.dept_id%TYPE;
  CURSOR cursor1 IS
    SELECT * FROM department;
  CURSOR cursor2 IS
    SELECT * FROM employee WHERE dept_id=did;
BEGIN
  FOR record1 IN cursor1
  LOOP
    did:=record1.dept_id;
    DBMS_OUTPUT.PUT_LINE('Department: ' || record1.dept_name);

    FOR record2 IN cursor2
    LOOP
      DBMS_OUTPUT.PUT_LINE(record2.name || ' ' || record2.salary);
    END LOOP;
  END LOOP;
END;
/
```

Output:

Statement processed.
Department: IT
Shyam 55000
Department: Finance
Ram 25000
Gita 34000